State

State · Order lifecycle NEW→PAID→SHIPPED with illegal guards · Build with Waqar

Today's Hook

- Problem: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Pattern: State — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



State: Order lifecycle NEW→PAID→SHIPPED with illegal guar

Real-World Scenario

- Running example: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.state;

import com.buildwithwaqar.patterns.gof.behavioral.state.OrderStateMachine;
/** Problem: enforce order lifecycle transitions and reject illegal ones. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== State problem: order lifecycle ===");
    var o = new OrderStateMachine();
    System.out.println(o.status());
    o.pay(); System.out.println("paid=" + o.status());
    o.ship(); System.out.println("shipped=" + o.status());
    try { o.cancel(); } catch (IllegalStateException e) { System.out.println("illegalCancel=caught"); }
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.state;

public final class OrderStateMachine {
    private State state = new NewState();

    public String status() { return state.name(); }
    public void pay() { state = state.pay(this); }
    public void ship() { state = state.ship(this); }
    public void cancel() { state = state.cancel(this); }

    private sealed interface State permits NewState, PaidState, ShippedState, CancelledState {
        String name();
        default State pay(OrderStateMachine ctx) { throw new IllegalStateException(name()); }
        default State ship(OrderStateMachine ctx) { throw new IllegalStateException(name()); }
        default State cancel(OrderStateMachine ctx) { throw new IllegalStateException(name()); }
    }

    private static final class NewState implements State {
        public String name() { return "NEW"; }
        public State pay(OrderStateMachine ctx) { return new PaidState(); }
        public State cancel(OrderStateMachine ctx) { return new CancelledState(); }
    }
```

Solution (2)

```
    private static final class PaidState implements State {
        public String name() { return "PAID"; }
        public State ship(OrderStateMachine ctx) { return new ShippedState(); }
        public State cancel(OrderStateMachine ctx) { return new CancelledState(); }
    }
    private static final class ShippedState implements State {
        public String name() { return "SHIPPED"; }
    }
    private static final class CancelledState implements State {
        public String name() { return "CANCELLED"; }
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.state;

import com.buildwithwaqar.patterns.gof.behavioral.state.OrderStateMachine;
/** Problem: enforce order lifecycle transitions and reject illegal ones. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== State problem: order lifecycle ===");
    var o = new OrderStateMachine();
    System.out.println(o.status());
    o.pay(); System.out.println("paid=" + o.status());
    o.ship(); System.out.println("shipped=" + o.status());
    try { o.cancel(); } catch (IllegalStateException e) { System.out.println("illegalCancel=caught"); }
  }
}
```

Walkthrough Output

```
=== State problem: order lifecycle ===
NEW
paid=PAID
shipped=SHIPPED
illegalCancel=caught
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with State.

Related Patterns

- Contrast State with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → State → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.